

Chapter 1 – Sorting Algorithm

1.1 Purpose of Task A

The purpose of Task A is to select the highest-priority inspection locations from three candidate datasets. Each candidate location has a location ID and a priority score. In this project, a higher priority score means that the location should be inspected earlier, so the program needs to rank the locations correctly before selecting the inspection targets.

To complete this part, three sorting algorithms were implemented and compared: Bubble Sort, Quick Sort, and Merge Sort. After each dataset was sorted, the top 10 locations were selected from Dataset A, Dataset B, and Dataset C. These 30 selected locations were then used in the graph algorithm part of the project as the important inspection targets.

1.2 Dataset and Ranking Rule

The program reads three CSV files: `candidates_A.csv`, `candidates_B.csv`, and `candidates_C.csv`. Each row contains two fields: `location_id` and `priority_score`. The data were stored as `Location` objects in an `ArrayList`, which made it convenient to sort the locations and then take the first 10 results.

All three sorting algorithms use exactly the same ranking rule. First, the locations are sorted by `priority_score` in descending order. If two locations have the same `priority_score`, they are sorted by `location_id` in ascending order. This second rule is important because it makes the output deterministic instead of leaving locations with equal scores in an unclear order.

1.3 Sorting Algorithms Implemented

Bubble Sort was implemented because it is simple and easy to trace. It repeatedly compares neighbouring locations and swaps them if they are in the wrong order. An early-stop flag was also used, so the algorithm can finish early if a pass makes no swaps. However, its average and worst-case time complexity is $O(n^2)$, so it is normally not suitable for large datasets.

Quick Sort was implemented using recursion and partitioning. In this implementation, the last element of the current range is used as the pivot. Locations that should appear before the pivot are moved to the left side, and the remaining locations stay on the right side. Quick Sort has an average time complexity of $O(n \log n)$, but its performance can become worse when the pivot choice creates unbalanced partitions.

Merge Sort was implemented using the divide-and-conquer idea. The list is divided into two smaller lists, each part is sorted recursively, and then the two sorted lists are merged. Merge Sort has $O(n \log n)$ time complexity in both average and worst cases, so its running time is usually more stable. The main trade-off is that it needs extra memory for temporary lists during the merge process.

1.4 Experimental Method

For each dataset, the program first loads the original CSV file into an `ArrayList<Location>`. Before every sorting run, the dataset is read again from the file so that each algorithm sorts the same original input rather than a list that has already been sorted by another algorithm. This makes the comparison fairer.

Each sorting algorithm was run three times on each dataset. The program used `System.nanoTime()` to measure the running time and then converted the average time

into milliseconds. After sorting, the first 10 locations in the sorted list were recorded as the selected inspection targets. The output with priority scores was also checked to confirm that the ranking rule was correctly applied.

1.5 Sorting Results

Dataset	Bubble Sort Avg Time	Quick Sort Avg Time	Merge Sort Avg Time	Top 10 Selected Locations
Dataset A	0.3646 ms	7.2063 ms	0.9136 ms	L0001, L0002, L0003, L0004, L0005, L0006, L0007, L0008, L0009, L0010
Dataset B	10.2005 ms	0.0783 ms	0.3961 ms	L0101, L0102, L0103, L0104, L0105, L0106, L0107, L0108, L0109, L0110
Dataset C	0.9594 ms	0.4427 ms	0.4790 ms	L0201, L0202, L0203, L0204, L0205, L0206, L0207, L0208, L0209, L0210

In the output, the three algorithms gave the same Top 10 locations for each dataset, which shows that the sorting results were consistent. I also checked the version of the output with priority scores. Dataset A and Dataset B were already in descending `priority_score` order in the selected results. Dataset C was useful for checking the second rule, because the Top 10 locations all had the same score of 5000.0, and the result was ordered from L0201 to L0210 by `location_id`.

1.6 Performance Analysis

The results show that the input order has a clear effect on the observed running time.

For Dataset A, Bubble Sort had the shortest running time. At first this looks different from the usual expectation, because Bubble Sort is normally slower than Quick Sort and Merge Sort. However, Dataset A was already close to the required descending order.

Since my Bubble Sort used an early-stop flag, it could finish quickly when no swaps were needed. My Quick Sort used the last element as the pivot, so it was not ideal for this nearly ordered input.

For Dataset B, Quick Sort was the fastest, while Bubble Sort was clearly slower. This result matches the normal idea of the algorithms better. Bubble Sort has to compare neighbouring elements many times, so its $O(n^2)$ cost becomes more obvious when the data is not already in a good order. Quick Sort divided the list by partitioning, so it finished much faster.

For Dataset C, Quick Sort was the fastest and Merge Sort was close behind. The more important point is that the Top 10 locations had the same `priority_score`. Because of this, the program had to use `location_id` as the tie-breaking rule. The result from L0201 to L0210 shows that this rule was handled correctly.

In this experiment, Merge Sort looked more consistent than the other two algorithms. Its running time changed less dramatically compared with Bubble Sort and Quick Sort. This also matches the theory, because Merge Sort keeps $O(n \log n)$ time complexity in both average and worst cases. The main disadvantage is that it uses extra memory during merging.

If the number of candidate locations became much larger, Bubble Sort would not be a good choice because $O(n^2)$ grows too quickly. Quick Sort and Merge Sort would both be more suitable for large input sizes. Quick Sort may be faster in practice, but its performance depends on pivot choice. Merge Sort is safer when stable runtime is more important.

1.7 Algorithm Choice for the Final System

If I had to choose only one sorting algorithm for the final system, I would choose Merge Sort. My reason is that the data may come from different systems, so the original order may not always be predictable. Merge Sort is not always the fastest in my results, but it

gives more stable $O(n \log n)$ performance, which is useful for this kind of system. Therefore, I chose runtime stability rather than the simplest implementation as the main consideration.

However, if memory usage becomes a bigger issue, I would also consider Quick Sort. Quick Sort normally uses less extra memory than Merge Sort, and it can be very fast with a better pivot strategy. I would not choose Bubble Sort for a larger version of the system, because its $O(n^2)$ time complexity would become a problem when the number of locations increases.